

AWS INTERVIEW GUIDE - Complete

EventBridge, SQS, Lambda, S3 & Multi-Region Architecture *Date: 2026-09-17 | LMS Project*

TABLE OF CONTENTS

1. EventBridge Event-Driven Architecture
 2. SQS Queue Patterns
 3. S3 Storage & Presigned URLs
 4. API Gateway & Lambda Authorizer
 5. Lambda Configuration & Optimization
 6. IAM Roles & Policies
 7. Multi-Region Failover
 8. Terraform Infrastructure as Code
 9. Scaling Strategies
 10. Interview Scenarios & Q&A
-

SECTION 1: EVENTBRIDGE

What & Why EventBridge?

EventBridge = Serverless event bus for routing events between AWS services and applications.

Benefits:

1. **Decoupling:** Publishers unaware of consumers
2. **Multiple consumers:** Single event → multiple targets
3. **Cross-account:** Events from Course Management account
4. **Filtering:** EventBridge matches patterns before invoking
5. **Built-in retry:** Exponential backoff + DLQ support

```
Course Management Service
  ↓ PutEvents (lesson.status)
  ↓
EventBridge Custom Bus
├─ Event pattern: { source: "lesson.status", detail-type: "LessonUpdate" }
├─ Target 1: Lambda (direct invocation)
└─ Target 2: SQS Queue (ECS polls)
```

Event Pattern Matching

```
{
  "source": ["lesson.status"],
```

```

    "detail-type": ["LessonUpdate"],
    "detail": {
      "status": ["completed", "failed"]
    }
  }
}

```

Matching Logic:

- AND within same level (all must match)
- OR across array values (any value matches)

Matching Events:

```

{
  "source": "lesson.status",
  "detail-type": "LessonUpdate",
  "detail": {
    "lessonId": "lesson-123",
    "status": "completed"
  }
}
✓ MATCHES

```

Non-Matching Events:

```

{
  "source": "lesson.status",
  "detail-type": "LessonDeleted", // ← doesn't match
  "detail": { "status": "deleted" }
}
✗ DOESN'T MATCH

```

Lambda vs ECS Runtime

Lambda Runtime

```

resource "aws_cloudwatch_event_target" "lambda_target" {
  rule      = aws_cloudwatch_event_rule.lesson_update_rule.name
  target_id = "SendToLambda"
  arn       = module.lms_backend.lambda_function_arn:alias/current-version
  role_arn  = aws_iam_role.eventbridge_invoke_lambda.arn
}

```

Characteristics:

- Direct EventBridge → Lambda invocation
- Cold start: 5-10 seconds (Java/Spring Boot)
- Latency: 0-5s end-to-end
- Cost: Pay per invocation + duration
- Scaling: Automatic (default 1000 concurrent)
- Best for: Bursty, unpredictable traffic

Example flow:

```

EventBridge receives lesson.status event
↓ (0.1s)
Lambda cold start (5s if not warm)
↓
Spring Boot context init (2-3s)
↓
Process event (1-2s)
↓
Update DynamoDB (0.1s)
= 8-11s total (first time), 100-200ms (subsequent)

```

ECS Runtime

```

resource "aws_cloudwatch_event_target" "sqs_target" {
  rule      = aws_cloudwatch_event_rule.lesson_update_rule.name
  target_id = "SendToSqs"
  arn       = aws_sqs_queue.event_queue.arn
  role_arn  = aws_iam_role.eventbridge_send_sqs.arn
}

# ECS task polls SQS continuously
while (true) {
  messages = sqs.receiveMessages(queueUrl, maxMessages=10);
  messages.forEach(msg -> processEvent(msg));
  sqs.deleteMessages(msg);
  Thread.sleep(1000); // Poll interval
}

```

Characteristics:

- EventBridge → SQS → ECS task
- Cold start: 0 (container always running)
- Latency: 5-15s (queue polling + processing)
- Cost: Fixed (regardless of traffic)
- Scaling: Manual task count + auto-scaling
- Best for: Sustained, predictable traffic

Example flow:

```

EventBridge receives event
↓ (0.1s)
SQS stores message
↓ (1s - polling interval)
ECS task receives message
↓ (1-2s)
Process event (no cold start!)
= 2-3s total (consistent)

```

EventBridge Scheduler (Cron)

```

resource "aws_scheduler_schedule" "survey_mail_scheduler" {
  name          = "prod-survey-mail-schedule"
  schedule_expression = "cron(0 9 ? * MON-FRI)" # 9 AM weekdays
  timezone      = "Asia/Kolkata"

  target {
    arn      = module.lms_backend.lambda_function_arn:alias/current
    role_arn = aws_iam_role.scheduler_invoke_lambda.arn
    input    = jsonencode({ source = "aws.scheduler", job = "survey-mail" })
  }
}

resource "aws_scheduler_schedule" "leaderboard_rebuild" {
  name          = "prod-leaderboard-schedule"
  schedule_expression = "cron(0 2 * * ? *)" # 2 AM daily

  target {
    arn      = module.lms_backend.lambda_function_arn:alias/current
    input    = jsonencode({ source = "aws.scheduler", job = "assembly-leaderboard" })
  }
}

```

Handler (Java):

```

public Void handleRequest(Object event, Context context) {
    Map<String, Object> eventMap = (Map<String, Object>) event;
    String source = (String) eventMap.get("source");

    if ("aws.scheduler".equalsIgnoreCase(source)) {
        String job = (String) eventMap.get("job");
        if ("assembly-leaderboard".equalsIgnoreCase(job)) {
            AssemblyLeaderboardService.rebuild();
        } else if ("survey-mail".equalsIgnoreCase(job)) {
            SurveyMailService.sendEmails();
        }
    }
}

```

```
    }  
  }  
  return null;  
}
```

SECTION 2: SQS (Simple Queue Service)

Queue Configuration

```
resource "aws_sqs_queue" "event_dlq" {  
  name = "lms-event-dlq"  
  message_retention_seconds = 1209600 # 14 days  
}  
  
resource "aws_sqs_queue" "event_queue" {  
  name = "lms-event-queue"  
  visibility_timeout_seconds = 300 # 5 minutes  
  message_retention_seconds = 345600 # 4 days  
  receive_wait_time_seconds = 20 # Long polling  
  
  redrive_policy = jsonencode({  
    deadLetterTargetArn = aws_sqs_queue.event_dlq.arn  
    maxReceiveCount = 3 # After 3 failures → DLQ  
  })  
}
```

Message Lifecycle

Happy Path:

```
Time  Event  
0:00  EventBridge sends message to SQS  
0:05  ECS polls, receives message  
0:06  Message becomes invisible (300s visibility timeout)  
0:10  Task processes event (write to DynamoDB)  
0:15  Task calls deleteMessage(ReceiptHandle)  
0:16  Message removed from queue  
Result: Message processed once ✓
```

Failure Path (Task Crashes):

```
Time  Event  
0:00  EventBridge sends message  
0:05  ECS polls, receives message  
0:06  Message becomes invisible
```

```
0:10 Task starts processing
0:11 Container OOM crash ✨
0:12 ECS detects crash, restarts container
5:06 Visibility timeout expires → message reappears
5:10 New ECS task polls, receives same message (receive_count=2)
5:15 Task processes again
5:16 Message deleted
Result: Message processed twice (need idempotency) ⚠
```

After Max Retries:

```
Attempt 1: Fails (visibility timeout)
Attempt 2: Fails (visibility timeout)
Attempt 3: Fails (visibility timeout)
Attempt 4: receive_count exceeds maxReceiveCount(3)
           → Message moves to Dead-Letter Queue

DLQ: Message retained 14 days for investigation
Engineer: Inspect, fix, manually replay if needed
```

Visibility Timeout Explained

Too short (60s):

- Fast processing: OK
- Slow processing (>60s): Redelivered immediately
- Result: Duplicate processing, wasted cost

Too long (900s):

- Failed message waits long before retry
- Slow feedback loop for debugging
- Resource tied up

Our choice (300s):

- Matches Lambda timeout
- Allows batch processing multiple events
- Reasonable retry window

Dead-Letter Queue (DLQ)

```
// Message that causes parsing error:
{ "invalid": json }

// Attempt 1-3: Parse fails, visibility timeout → reappear
// Attempt 4: Move to DLQ
```

```
// Engineer investigates DLQ:
Message: { "invalid": json }
Error: SyntaxError: Unexpected token
Action:
  1. Add validation for malformed JSON
  2. Fix consumer code
  3. Manually replay message or discard

// Monitor DLQ
CloudWatch Alarm: if DLQ message count > 0, alert
```

SECTION 3: S3 STORAGE

S3 Event Notifications

```
resource "aws_s3_bucket_notification" "s3_event_notification" {
  bucket = "lms-media-bucket"

  lambda_function {
    lambda_function_arn = module.lms_s3_media_listener.lambda_arn
    events               = ["s3:ObjectCreated:*"]
    filter_prefix        = "media/" # Only media/ folder
    filter_suffix        = ".mp4"   # Only MP4 files
  }
}
```

Triggered Events:

- `s3:ObjectCreated:Put` - Standard upload
- `s3:ObjectCreated:Post` - HTML form upload
- `s3:ObjectCreated:Copy` - Copy operation
- `s3:ObjectCreated:CompleteMultipartUpload` - Large upload

S3 Event Handler

```
@Component
@Slf4j
public class S3MediaListenerHandler implements RequestHandler<S3Event, String> {

    @Override
    public String handleRequest(S3Event event, Context context) {
        // event.getRecords() contains:
        // - bucket name
        // - object key (full path)
        // - object size
        // - upload timestamp
    }
}
```

```
List<String> importList = mediaUtil.importJobs(event);

if (!importList.isEmpty()) {
    mediaService.saveServiceOrder(importList, context);
}

return "Processed " + importList.size() + " media files";
}
}
```

Presigned URLs for Secure Access

Problem: User needs to download private video from S3

- Can't expose credentials to client
- Can't trust client with AWS keys

Solution: Presigned URL:

```
// Backend generates (has AWS credentials)
const s3Client = new S3Client({ region: "us-east-1" });
const command = new GetObjectCommand({
    Bucket: "lms-media-bucket",
    Key: "videos/course-123/lecture.mp4"
});

const presignedUrl = await getSignedUrl(s3Client, command, {
    expiresIn: 3600 // 1 hour
});

// URL contains:
// https://lms-media-bucket.s3.amazonaws.com/videos/course-123/lecture.mp4
// ?X-Amz-Algorithm=AWS4-HMAC-SHA256
// &X-Amz-Credential=AKIA...%2Fus-east-1%2Fs3%2Faws4_request
// &X-Amz-Date=20260917T093000Z
// &X-Amz-Expires=3600
// &X-Amz-SignedHeaders=host
// &X-Amz-Signature=abcd1234...
```

Multi-Tenant Isolation:

```
String tenantId = getTenantFromAuth(); // "TENANT#cognizant"
String courseId = request.getCourseId(); // "course-123"

// S3 key includes tenant prefix
String s3Key = "/videos/" + tenantId + "/course-" + courseId + "/lecture.mp4";
```

```
String presignedUrl = s3Client.generatePresignedUrl(bucket, s3Key, expiry);  
// → Only cognizant tenant can access (signature includes key path)
```

SECTION 4: API GATEWAY & LAMBDA AUTHORIZER

Lambda Authorizer Flow

```
// Input: API Gateway sends event  
{  
  type: "TOKEN",  
  authorizationToken: "Bearer eyJhbGc...",  
  methodArn: "arn:aws:execute-api:ap-south-1:123:api/stage/POST/courses"  
}
```

Step 1: Extract tenant from Origin:

```
function extractOriginHost(event) {  
  const origin = event.headers?.["Origin"] || event.headers?.["origin"];  
  const host = new URL(origin).hostname; // "dev.lms.cognizant.com"  
  return host;  
}
```

Step 2: Load tenant configuration (cached):

```
async function loadTenantConfig() {  
  const now = Date.now();  
  // 5-minute cache  
  if (cache.tenantConfig && (now - cache.tenantConfig.cachedAt) < 5 * 60 * 1000) {  
    return cache.tenantConfig.data;  
  }  
  
  // Load from S3  
  const response = await s3.send(new GetObjectCommand({  
    Bucket: "lms-config-bucket",  
    Key: "tenant-config.json"  
  }));  
  
  const config = JSON.parse(await response.Body.transformToString());  
  cache.tenantConfig = { data: config, cachedAt: now };  
  return config;  
}
```

Step 3: Get tenant-specific Cognito pool:

```

async function getTenantConfig(host) {
  const cfg = await loadTenantConfig();

  // hosts: { "dev.lms.cognizant.com": { userPoolId, clientId } }
  const hostEntry = cfg.hosts?.[host];
  if (!hostEntry) throw new Error(`Unknown host: ${host}`);

  return {
    userPoolId: hostEntry.userPoolId,
    clientId: hostEntry.clientId,
    region: hostEntry.region || "ap-south-1"
  };
}

```

Step 4: Verify JWT signature:

```

async function verifyJWT(token, userPoolId, clientId) {
  const key = `${userPoolId}|${clientId}`;

  // Cache verifier per tenant (expensive to create)
  if (!cache.verifiers.has(key)) {
    cache.verifiers.set(key, CognitoJwtVerifier.create({
      userPoolId: userPoolId,
      clientId: clientId,
      tokenUse: "id"
    }));
  }

  const verifier = cache.verifiers.get(key);
  return await verifier.verify(token); // Throws if invalid
}

```

Step 5: Return AuthPolicy:

```

return {
  principalId: claims.sub, // user-123
  policyDocument: {
    Version: "2012-10-17",
    Statement: [{
      Action: "execute-api:Invoke",
      Effect: "Allow",
      Resource: "arn:aws:execute-api:*"
    }]
  },
  context: {
    tenantId: "TENANT#cognizant",
    userId: "user-123",
    roles: "ADMIN,AUTHOR"
  }
}

```

```
}
};
```

Why Cache JWT Verifiers?

Without caching:

Request 1: `CognitoJwtVerifier.create()` → fetch JWKS (500ms)
 → verify token (50ms)
 = 550ms total

Request 2 (same tenant): Same flow (550ms)

Throughput: ~1 request/second 🐢

With caching:

Request 1: `CognitoJwtVerifier.create()` (500ms)
 → `cache[userPoolId|clientId] = verifier`
 → verify (50ms)
 = 550ms

Request 2 (same tenant): Cache hit (1ms)
 → verify (50ms)
 = 51ms

Request 3 (different tenant): Cache miss (500ms)
 → verify (50ms)
 = 550ms

Throughput: ~100 requests/second with ~2 tenant pools 🚀

SECTION 5: LAMBDA CONFIGURATION

Lambda Environment Variables

```
locals {
  lambda_environment_variables = {
    # DynamoDB
    AWS_DYNAMODB_COURSE_ENROLLMENT_TABLE = "prod-lms-course-enrollment-table"
    AWS_DYNAMODB_ASSEMBLY_STATS_TABLE    = "prod-lms-assembly-stats-table"

    # Cognito
    AWS_COGNITO_CLIENT_ID =
    data.aws_secretsmanager_secret_version.fetched_secret.secret_string["AWS_COGNITO_C
```

```

    LIENT_ID"]
    AWS_COGNITO_USER_POOL_ID =
data.aws_secretsmanager_secret_version.fetched_secret.secret_string["AWS_COGNITO_U
SER_POOL_ID"]

    # Services
    COURSE_INT_SERVICE = "https://courseintegration.${var.hosted_zone_name}"
    USER_INT_SERVICE   = "https://userintegration.${var.hosted_zone_name}"

    # EventBridge
    COURSE_MNGMNT_BUS_ARN =
"arn:aws:events:${var.aws_region}:${var.course_management_account}:event-bus/..."

    # SQS (cross-account)
    SQS_URL =
"https://sqs.${var.aws_region}.amazonaws.com/${var.shared_services_account}/email-
queue"
  }
}

```

Provisioned Concurrency

```

resource "aws_lambda_provisioned_concurrent_executions" "current" {
  function_name           = module.lms_backend.lambda_function_name
  provisioned_concurrent_executions = 20 # Reserve 20 warm instances
}

```

Cost: \$0.015/hour per unit $\approx$ \$109/month per unit (always-on)

Benefit: No cold starts (100ms response vs 5-10s)

Trade-off:

- Provisioned: Fixed cost + bursts auto-scale
- On-demand: Variable cost, cold starts for bursts

Memory & CPU Scaling

```

Memory → CPU (linear)
128 MB   → 1/8 vCPU
256 MB   → 1/4 vCPU
512 MB   → 1/2 vCPU
1024 MB  → 1 vCPU
3008 MB  → 3 vCPU (max)

```

Example optimization:

- Task: Sort 100k items (CPU-bound)
 - With 512 MB: 10 seconds
 - With 2 GB: 2.5 seconds (4x faster, 4x memory cost)
 - Faster often cheaper (shorter billing duration)
-

SECTION 6: IAM ROLES & POLICIES

Lambda Execution Role

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "CloudWatchLogs",
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ],
      "Resource": "arn:aws:logs:*:*:*"
    },
    {
      "Sid": "DynamoDBRead",
      "Effect": "Allow",
      "Action": [
        "dynamodb:GetItem",
        "dynamodb:Query",
        "dynamodb:Scan",
        "dynamodb:BatchGetItem"
      ],
      "Resource": [
        "arn:aws:dynamodb:ap-south-1:123:table/lms-*"
      ]
    },
    {
      "Sid": "DynamoDBWrite",
      "Effect": "Allow",
      "Action": [
        "dynamodb:PutItem",
        "dynamodb:UpdateItem",
        "dynamodb>DeleteItem",
        "dynamodb:BatchWriteItem"
      ],
      "Resource": "arn:aws:dynamodb:ap-south-1:123:table/lms-*"
    },
    {
      "Sid": "S3Access",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
```

```

        "s3:PutObject",
        "s3:ListBucket"
    ],
    "Resource": [
        "arn:aws:s3:::lms-config-bucket",
        "arn:aws:s3:::lms-config-bucket/*"
    ]
},
{
    "Sid": "SecretsManagerRead",
    "Effect": "Allow",
    "Action": "secretsmanager:GetSecretValue",
    "Resource": "arn:aws:secretsmanager:ap-south-1:999:secret:lms/*"
}
]
}

```

Cross-Account STS Assume Role

```

StsClient stsClient = StsClient.builder().region(Region.of(region)).build();

StsAssumeRoleCredentialsProvider credentialsProvider =
    StsAssumeRoleCredentialsProvider.builder()
        .refreshRequest(r -> r
            .roleArn("arn:aws:iam::222222222222:role/CrossAccountEmailRole")
            .roleSessionName("LMSEmailAccess")
            .durationSeconds(3600))
        .stsClient(stsClient)
        .build();

SqsClient sqsClient = SqsClient.builder()
    .credentialsProvider(credentialsProvider)
    .region(Region.of(region))
    .build();

```

Result: Lambda assumes role in email service account, accesses their SQS queue.

SECTION 7: MULTI-REGION FAILOVER

Architecture

Primary Region (ap-south-1)	Failover Region (us-east-1)
├ API Gateway	├ API Gateway
├ Lambda	├ Lambda
├ DynamoDB (primary)	├ DynamoDB (replica, read-only)
├ EventBridge	├ EventBridge

```
└─ SQS
└─ Route53 Health Check → Active
```

```
└─ SQS
└─ Standby (DNS weighted=0)
```

Failure Scenario

Normal Operation:

Route53: ap-south-1 weight=100, us-east-1 weight=0
All traffic → ap-south-1

Disaster:

Time 0:00 - Earthquake in Mumbai 🌍
Time 0:10 - API Gateway health check fails
Time 1:30 - Route53 failover triggers (after 3 failed checks)
- Changes weights: ap-south-1=0, us-east-1=100

Time 1:35 - DNS propagates (TTL=60s)
Time 1:40 - Clients resolve to us-east-1

Traffic now → us-east-1 (Lambda + DynamoDB replica)

Data Loss?

DynamoDB replica is read-only → promotes to primary table
EventBridge events in ap-south-1 might be lost (no replay)
SQS messages in ap-south-1 lost (regional service)

Recovery:

ap-south-1 recovers
→ Terraform rebuild
→ DynamoDB replication restored
→ Route53 weighted back to primary
→ Failback complete

SECTION 8: TERRAFORM INFRASTRUCTURE AS CODE

Multi-Environment Directory Structure

```
lms-ai-service/cicd/
└─ cognizant/
   └─ prod/terraform/
      ├── backend.tf      (S3 state: cognizant/prod/tfstate)
      ├── providers.tf
      ├── variables.tf
      ├── main.tf         (module instantiation)
      ├── iam.tf
      ├── eventbridge.tf
      └── locals.tf
   └─ stage/terraform/
```

```
└─ community/
   └─ dev/terraform/
   └─ qa/terraform/
   └─ stage/terraform/
   └─ prod/terraform/
```

Each environment has independent tfstate → No cross-environment affects

Backend & Provider Setup

```
# backend.tf
terraform {
  backend "s3" {
    bucket      = "lms-terraform-state"
    key         = "lms-ai-service/cognizant/prod/terraform.tfstate"
    region      = "ap-south-1"
    encrypt     = true
    dynamodb_table = "terraform-locks" # Prevent concurrent applies
  }
}

# providers.tf
provider "aws" {
  alias   = "primary"
  region = var.aws_region
  assume_role {
    role_arn = "arn:aws:iam::${var.aws_account_id}:role/TerraformExecutionRole"
  }
}

provider "aws" {
  alias   = "failover"
  region = var.aws_failover_region
  assume_role {
    role_arn = "arn:aws:iam::${var.aws_account_id}:role/TerraformExecutionRole"
  }
}

provider "aws" {
  alias   = "sharedservices"
  region = var.aws_region
  assume_role {
    role_arn =
"arn:aws:iam::${var.aws_shared_services_account_id}:role/CrossAccountRole"
  }
}
```

Module Usage

```
module "lms_backend" {
  source = "github.com/Cognizant-SPE/lms-terraform-modules//microservices-
terraform?ref=v0.6.0.9"

  # Inputs
  specfile           = var.api_specfile_path
  api_name_prefix    = local.api_name_prefix
  lambda_function_memory = var.lambda_function_memory
  lambda_function_timeout = var.lambda_function_timeout
  provisioned_concurrent_executions = var.provisioned_concurrency

  lambda_enable_vpc      = true
  use_existing_vpc       = true
  existing_vpc_id        = data.aws_vpc.existing.id

  # Multi-region
  configure_failover      = true
  aws_failover_region     = var.aws_failover_region

  # Outputs
  # module.lms_backend.lambda_function_arn
  # module.lms_backend.lambda_role_name
  # module.lms_backend.api_endpoint
}
```

SECTION 9: SCALING STRATEGIES

Horizontal Scaling (Add More Instances)

Baseline: 1 Lambda, handles 100 req/s
Load increases to 500 req/s

Solution: AWS auto-scales to 5+ concurrent executions
Result: Each Lambda handles $\sim 100 \text{ req/s} \times 5 = 500 \text{ req/s}$
Cost: Auto-scaled instances only pay-per-invocation

Vertical Scaling (More Powerful Single Instance)

Single Lambda 512 MB: Sorts 100k items in 10s
Upgrade to 2048 MB (4x CPU): Sorts in 2.5s
Cost: 4x memory = 4x monthly cost (if provisioned)
But execution 4x faster → shorter billed duration

DynamoDB Scaling

Option 1: PAY_PER_REQUEST (auto-scale)

Cost: \$1.25 per million requests

Scale: Instant (no tuning needed)

Best: Unpredictable workloads

Option 2: PROVISIONED + AUTO-SCALING

Cost: Fixed WCU/RCU + auto-scaling above baseline

Scale: Minutes (capacity increase)

Best: Predictable baseline

This system uses: PAY_PER_REQUEST (simplest, event-driven workload)

10x Traffic Scaling Plan

Current:

Lambda: 10 provisioned concurrency units

DynamoDB: Pay-per-request (peak: 1000 req/s)

Cost: ~\$1000/month

10x Traffic (10,000 req/s):

Lambda: Increase provisioned concurrency to 100 units

Cost: +\$800/month

DynamoDB: Scaling stays same (pay-per-request auto-scales)

Cost: +\$10k/month (10x requests)

Caching: Add Redis cluster

Cost: +\$500/month (reduce DynamoDB reads 50%)

Database: Add GSI for hot queries

Cost: +\$200/month

Total Cost Increase: ~\$11.5k/month

Strategies: Caching, indexing, query optimization

SECTION 10: INTERVIEW SCENARIOS

Q: Design EventBridge for cross-account event delivery

Requirements:

- Course Management (Account A) publishes events
- LMS (Account B) consumes events

Architecture:

Account A: Course Management Service

```
→ eventBridgeClient.putEvents(  
    eventBusName: "arn:aws:events:...B:event-bus/lms-bus"
```

)

Account B: LMS

- Create custom event bus
- Grant Account A permission: events:PutEvents
- Create EventBridge rule on custom bus
- Targets: Lambda or SQS

Implementation:

```
# Account B: Custom Event Bus
resource "aws_cloudwatch_event_bus" "lms_bus" {
  name = "lms-events-bus"
}

# Account B: Grant permission to Account A
resource "aws_cloudwatch_event_permission" "publish_from_course_mgmt" {
  event_bus_name = aws_cloudwatch_event_bus.lms_bus.name
  principal      = "111111111111" # Account A ID
  statement_id   = "PublishCourseMgmtEvents"
  action         = "events:PutEvents"
}

# Account B: Create rule on custom bus
resource "aws_cloudwatch_event_rule" "lesson_updates" {
  event_bus_name = aws_cloudwatch_event_bus.lms_bus.name
  event_pattern = jsonencode({
    source = ["course.management"]
    detail-type = ["LessonStatusChanged"]
  })
}

# Account B: Route to Lambda or SQS
resource "aws_cloudwatch_event_target" "lambda" {
  rule      = aws_cloudwatch_event_rule.lesson_updates.name
  event_bus_name = aws_cloudwatch_event_bus.lms_bus.name
  arn       = aws_lambda_function.enrollment_listener.arn
  role_arn  = aws_iam_role.eventbridge_invoke.arn
}
```

Q: Walk through a multi-region failover

Scenario: Primary region (ap-south-1) becomes unavailable

Step-by-step:

1. **Health check fails** (Route53 every 30s):
 - API Gateway in ap-south-1 returns 503

- Route53 marks as unhealthy

2. **Failover triggers** (~30-60s after detection):

- Route53 weighted routing: ap-south-1=0, us-east-1=100
- New requests resolve to us-east-1 IP

3. **DNS propagates** (TTL=60s):

- Cached DNS entries expire
- Clients get new IP for us-east-1 endpoint

4. **Failover region active:**

- Lambda in us-east-1 starts processing
- DynamoDB replica becomes writable (automatic)
- EventBridge in us-east-1 active

5. **Potential data loss:**

- Events pending in ap-south-1 bus: Lost
- SQS messages in ap-south-1: Lost (regional service)
- In-flight Lambda invocations: Interrupted

6. **Recovery:**

- ap-south-1 repairs
- Terraform rebuild infrastructure
- DynamoDB replication re-established
- Route53 fails back (ap-south-1=100, us-east-1=0)

KEY INTERVIEW TIPS

1. **EventBridge**: Understand pattern matching, Lambda vs ECS runtimes, retry behavior
2. **SQS**: Know visibility timeout, DLQ, deduplication, long polling
3. **Presigned URLs**: How they enable secure S3 access without AWS credentials
4. **Lambda Authorizer**: Cognito JWT verification, per-tenant pools, caching importance
5. **Multi-region**: Route53 health checks, automatic failover, DynamoDB replication
6. **Scaling**: Horizontal (auto-scale), Vertical (memory), Database (on-demand vs provisioned)
7. **Terraform**: State management, modules, multi-region setup with providers
8. **IAM**: Least privilege, cross-account roles, resource-based policies

Study this with Frontend and Backend guides for comprehensive interview prep